LAB PROGRAMS

Compiler Design & Computer Networks

PROGRAM 1 — WEEK 10

Implement on a data set of characters the three CRC polynomials — CRC-12, CRC-16 and CRC-CCITT.

The program takes a character string from the user, converts it to binary, performs XOR-based CRC division, and displays the CRC and transmitted codeword.

C++ Program

```
#include <iostream>
#include <string>
using namespace std;

string toBinary(string s)
{
    string bin = "";

    for (char c : s)
    {
        for (int i = 7; i >= 0; i--)
            bin += ((c >> i) & 1) ? '1' : '0';
    }

    return bin;
}

string calculateCRC(string data, string poly)
{
    int n = poly.length();
    string temp = data + string(n - 1, '0');

    for (int i = 0; i <= temp.length() - n; i++)
    {
        if (temp[i] == '1')
        {
            for (int j = 0; j < n; j++)
            {
                if (temp[i + j] == poly[j])
                    temp[i + j] = '0';
                else
                    temp[i + j] = '1';
            }
        }
    }

    return temp.substr(temp.length() - n + 1);
}

int main()
{
    string str;

    cout << "Enter a string: ";
    cin >> str;

    string data = toBinary(str);

    cout << "\nBinary Data: " << data << endl;

    string crc12 = "1100000001111";
    string crc16 = "11000000000000101";
    string ccitt = "10001000000100001";

    string r12 = calculateCRC(data, crc12);
    string r16 = calculateCRC(data, crc16);
    string rccitt = calculateCRC(data, ccitt);

    cout << "\nCRC-12   : " << r12;
    cout << "\nCRC-16   : " << r16;
    cout << "\nCRC-CCITT: " << rccitt;

    cout << "\n\nTransmitted Data:";
    cout << "\nCRC-12   : " << data + r12;
    cout << "\nCRC-16   : " << data + r16;
    cout << "\nCRC-CCITT: " << data + rccitt;
```

```
    return 0;
}
```

PROGRAM 2 — WEEK 2, QUESTION 1

Design a Lexical Analyzer for the given language.

This pure C program ignores spaces, tabs, newlines and comments, and recognizes keywords, identifiers, numbers, operators and symbols. Identifier length is restricted to a reasonable value.

C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 20

char *keywords[] = {
    "int", "float", "char", "if", "else",
    "while", "for", "return", "void"
};

int isKeyword(char word[])
{
    int i;

    for (i = 0; i < 9; i++)
    {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main()
{
    FILE *fp;
    char filename[50];
    char ch, next;
    char token[100];
    int i;

    printf("Enter input file name: ");
    scanf("%s", filename);

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        printf("File cannot be opened.\n");
        return 0;
    }

    printf("\nTokens:\n");

    while ((ch = fgetc(fp)) != EOF)
    {
        if (ch == ' ' || ch == '\t' || ch == '\n')
            continue;

        if (ch == '/')
        {
            next = fgetc(fp);

            if (next == '/')
            {
                while ((ch = fgetc(fp)) != '\n' && ch != EOF);
                continue;
            }

            if (next == '*')
            {
                while (1)
                {
                    ch = fgetc(fp);

                    if (ch == '*' && fgetc(fp) == '/')
                        break;

                    if (ch == EOF)
                        break;
                }

                continue;
            }
```

```
            ungetc(next, fp);
            printf("Operator: /\n");
            continue;
        }

        if (isalpha(ch) || ch == '_')
        {
            i = 0;

            while (isalnum(ch) || ch == '_')
            {
                if (i < MAX)
                    token[i++] = ch;

                ch = fgetc(fp);
            }

            token[i] = '\0';
            ungetc(ch, fp);

            if (isKeyword(token))
                printf("Keyword: %s\n", token);
            else
                printf("Identifier: %s\n", token);

            continue;
        }

        if (isdigit(ch))
        {
            i = 0;

            while (isdigit(ch) || ch == '.')
            {
                token[i++] = ch;
                ch = fgetc(fp);
            }

            token[i] = '\0';
            ungetc(ch, fp);

            printf("Number: %s\n", token);
            continue;
        }

        if (ch == '+' || ch == '-' ||
            ch == '*' || ch == '%' ||
            ch == '=' || ch == '<' ||
            ch == '>')
        {
            next = fgetc(fp);

            if ((ch == '+' && next == '+') ||
                (ch == '-' && next == '-') ||
                (ch == '=' && next == '=') ||
                (ch == '<' && next == '=') ||
                (ch == '>' && next == '='))
            {
                printf("Operator: %c%c\n", ch, next);
            }
            else
            {
                printf("Operator: %c\n", ch);
                ungetc(next, fp);
            }

            continue;
        }

        if (ch == ';' || ch == ',' ||
            ch == '(' || ch == ')' ||
            ch == '{' || ch == '}' ||
            ch == '[' || ch == ']')
        {
            printf("Symbol: %c\n", ch);
            continue;
        }

        printf("Unknown: %c\n", ch);
    }

    fclose(fp);
    return 0;
}
```

PROGRAM 3 — WEEK 2, QUESTION 2

Implement the Lexical Analyzer using JLex, Flex, Lex or other lexical-analyzer-generating tools.

This is a separate Lex/Flex program. Save it as **lexer.l**.

Lex/Flex Program

```
%{
#include <stdio.h>
%}

%%

[ \t\n]+                    ;
"//".*                      ;
"/*"([^*]|\*+[^*/])*\*+"/"  ;

"int"|"float"|"char"|"if"|"else"|"while"|"for"|"return"|"void"
                            { printf("Keyword: %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*     { printf("Identifier: %s\n", yytext); }

[0-9]+(\.[0-9]+)?          { printf("Number: %s\n", yytext); }

"=="|"!="|"<="|">="|"++"|"--"
                            { printf("Operator: %s\n", yytext); }

"+"|"-"|"*"|"/"|"%"|"="|"<"|">"
                            { printf("Operator: %s\n", yytext); }

";"|","|"("|")"|"{"|"}"|"["|"]"
                            { printf("Symbol: %s\n", yytext); }

.                           { printf("Unknown: %s\n", yytext); }

%%

int main()
{
    printf("Enter source code (Ctrl+D to stop):\n\n");
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}
```

Execution commands

```
flex lexer.l
gcc lex.yy.c -o lexer
./lexer

On Windows:
lexer.exe
```